

Extended Abstract: The Decaboard Programming Toy

Decaboard is a programming toy designed for beginning programmers to practice basic programming concepts such as literals, variables, and expressions. It can also be used as a fun way to introduce shaders and GPUs to more experienced programmers, or simply as an easy way to make pretty patterns.

We used in a high school grade 12 programming course that was introducing Python and computer science. Students played with it about a quarter of the way through the course. Feedback was quite positive and all students found it at least some of it fun, and some students found it especially interesting as a way to get into shaders and graphics.

What is Decaboard?

Decaboard is based on GPU shaders. In a GPU shader you write a function that takes in the (x, y) position of a pixel in an image and returns a color. The one function is applied simultaneously to thousands of pixels in the image, and using modern GPUs this can be done fast enough to do real-time special effects in games and other graphical applications.

Decaboard is the same idea, except pixels are replaced by squares, and setting colors is replaced by setting the rotation angle of the square. You write a single function called `angleIt(row, col, elapsedTime)` that returns the angle of the square based only on the row, column, and how long the program has been running for.

For example, this code:

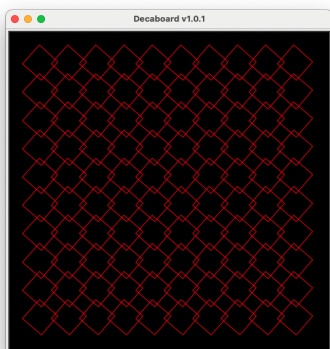
```
# example.py

import decaboard

def angleIt(row, col, elapsed_seconds):
    return 41

decaboard.run_board(angleIt)
```

Draws this:



Using it with Students

This can be used in a variety ways:

- With beginning programmers, you could give a short demo of what the program can do, then have students work through the problem set, or just play with it to see what patterns appear.
- For students who already know a little programming, you could introduce shaders and GPUs, and treat this as an introduction to shader-style programming. A PowerPoint presentation is provided that gives a brief overview of shaders and GPUs. Some students will likely have heard about shaders, or at least GPUs, from video games.
- For more advanced students, it could be the basis for project. For instance, they could use it to create visualizations of math functions, or they could modify [decaboard.py](#) to be have more squares, or to replace the square with pictures, etc. Another fun idea is to have the function return more than just the angle of the square, e.g. it could return a dictionary that contains the angle of the square, it's color, it's size, and it's position. This can generate a wide variety of interesting outputs.

Feedback from Students

Informally, all students seemed to enjoy the program, at least for the bit. The charm of so easily making interesting patterns seemed to really appeal to them. Some students were done after the problem sets, while a few other went it more deeply. The most interesting shader animations tend to use mathematical functions in clever ways.

Related Work

- [Design By Numbers](#) is an old idea for introducing programming to design-oriented students by providing a special language designed to set pixels on a 100-by-100 grid. It was soon eclipsed by [Processing](#), a more fully-featured library and language.
- [Replicube](#) is a 3D toy/game where you write shader-like programs to create interesting 3D shapes.